Chapter 24

Audio Synthesizer (WAV Generator): Algorithmic Sound Synthesis and RIFF/WAVE Binary Serialization

Shailesh (251150), Sirthik (251153)

24.1 Introduction and Problem Statement

Sound synthesis represents the mathematical intersection of physics, digital signal processing (DSP), and computer systems engineering. In an era where audio generation is often abstracted away behind monolithic third-party multimedia frameworks (such as SDL, PortAudio, or OpenAL), understanding the low-level generation of acoustic pressure waves from mathematical primitives provides fundamental insights into memory efficiency, real-time algorithms, and low-level binary data serialization.

The primary objective of this project is to architect and implement an extensible, decoupled, and highly performant **Audio Synthesizer and WAV Generator** in modern C++ (C++17). The system is engineered from the ground up without relying on external multimedia runtimes for audio generation or file formatting. Instead, the application directly generates discrete periodic waveforms, modulates their amplitude across temporal domains using an Attack-Decay-Sustain-Release (ADSR) envelope generator, and formats the synthesized raw pulse-code modulation (PCM) audio samples into a standard Microsoft Resource Interchange File Format (RIFF) WAVE audio file via unbuffered binary C++ file streams (`std::ofstream`).

The engineering requirements stipulated for the project comprise:

1. **Waveform Generation:** Mathematically generate three fundamental periodic acoustic waveforms: pure sinusoidal waves (*Sine*), odd-harmonic rich pulses (*Square*), and full harmonic-spectrum ramps (*Sawtooth*).

2. **Object-Oriented Architecture:** Implement an abstract base class `Oscillator` defining a pure virtual polymorphic interface for dynamic sample generation, alongside dedicated derived classes implementing concrete mathematical transfer functions.

3. **Temporal Dynamic Envelope:** Implement an encapsulated `Envelope` class providing four-stage piecewise linear ADSR modulation to prevent acoustic clicking artifacts caused by instantaneous step transients.

4. **Raw Binary Serialization:** Assemble canonical 44-byte RIFF/WAVE file headers and serialize 16-bit signed integer little-endian audio samples into persistent storage using formatted binary C++ file streams (`std::ofstream`).

5. **Defensive Validation:** Guard against acoustic aliasing by strictly enforcing the Nyquist-Shannon sampling theorem limit ($0 < f < f_s/2$) and clamping peak amplitude values to prevent integer overflow.

6. **Dual Frontend Interaction:** Deliver both a scriptable Command-Line Interface (CLI) supporting parameterized flags and an interactive Terminal User Interface (TUI) powered by FTXUI featuring real-time oscilloscopic waveform previews.

From an end-user perspective, the system provides sound designers, audio researchers, and students with an intuitive utility capable of generating calibrated acoustic test signals, musical tones, and customized synthesis sweeps exported directly into high-fidelity uncompressed 16-bit PCM `.wav` files compatible with any industry-standard media player or digital audio workstation (DAW).

24.2 Scope and Feature Set

The project delivers an end-to-end audio rendering pipeline partitioned into a static synthesis library (`synth_core`) and frontend driver executables. The feature set includes:

- **Feature 1 (Multi-Waveform Timbre Generation):** Algorithmic synthesis of three fundamental wave profiles:
  - *Sine Wave:* Pure fundamental frequency (f) devoid of harmonic overtones ($x(n) = \sin(2\pi\phi_n)$).
  - *Square Wave:* Composed exclusively of odd integer harmonics ($f, 3f, 5f, \dots$) decaying at $1/n$ amplitude ($x(n) = \text{sgn}(\sin(2\pi\phi_n))$).
  - *Sawtooth Wave:* Composed of both even and odd harmonics decaying at $1/n$ ($x(n) = 2\phi_n - 1$).
- **Feature 2 (Continuous Normalized Phase Accumulation):** A normalized phase accumulator $\phi \in [0.0, 1.0)$ updated as $\phi_{n+1} = (\phi_n + f/f_s) \pmod{1.0}$, maintaining sample-accurate phase tracking across arbitrary sampling frequencies (f_s).
- **Feature 3 (Piecewise Linear ADSR Dynamic Envelope):** A 4-stage time-domain amplitude modulator:
  - *Attack (T_A):* Linear ramp from zero amplitude to maximum gain (1.0).
  - *Decay (T_D):* Linear ramp down from peak amplitude to configured sustain level.
  - *Sustain (S):* Maintained steady-state amplitude plateau ($S \in [0.0, 1.0]$).
  - *Release (T_R):* Linear decay from sustain down to silence (0.0).
- **Feature 4 (Direct RIFF/WAVE Binary Stream Serialization):** Direct byte-level Little-Endian bit encoding and memory packing for the standard 44-byte canonical WAV header without relying on compiler-dependent struct packing.

- **Feature 5 (Defensive Boundary Validation and Nyquist Safety):** Enforces $0 < f < f_s/2$, clamps amplitude values to $[-1.0, 1.0]$ to prevent clipping, and validates non-negative envelope timing parameters.
- **Feature 6 (Dual Presentation Frontends):**
  - *CLI Executable (`synth`):* Optimized for rapid shell scripting and automated testing via standard flags.
  - *Interactive TUI (`synth-tui`):* Interactive terminal dashboard built using FTXUI featuring real-time Braille-canvas oscilloscopic previews.

24.2.1 Technology Stack

Table 24.1 provides a comprehensive overview of the technologies, tools, and libraries utilized across the engineering lifecycle.

Layer / Domain	Technology	Standard	License	Architectural Rationale
Primary Language	Modern C++	ISO/IEC C++17	Open ISO	Deterministic memory management (RAII), zero-overhead abstractions, compile-time type safety.
DSP Core	C++ Standard Library	C++17 STL	ISO	Contiguous sample buffers and fixed-width integer types without external third-party runtimes.
Binary File I/O	`std::ofstream`	C++ Stream	ISO	Unbuffered raw binary file output with manual little-endian byte serialization.
Build System	CMake	Version 3.14+	BSD 3-Cl.	Declarative target-based orchestration; partitions engine into static library and dual executables.
TUI Interface	FTXUI	Version 5.0.0	MIT	Declarative functional terminal UI providing interactive parameter sliders and keyboard controls.
Waveform Preview	`ftxui::Canvas`	FTXUI Comp.	MIT	Visualizer rendering continuous waveforms using Unicode Braille dot matrices.
Audio Format	Microsoft RIFF/WAVE	Canonical 1991	Open Spec	Uncompressed 16-bit linear PCM audio container with universal DAW and player support.
Verification	Audacity & Hex Dumps	v3.x / CLI	GPL v2 / OS	FFT spectrum analysis, envelope inspection, and hexadecimal verification of WAV headers.

Table 24.1: Comprehensive Technology Stack of the Audio-WAV-Synthesizer Project

24.3 Object-Oriented Architecture

The software architecture adheres rigorously to established Object-Oriented Programming (OOP) principles, guaranteeing high cohesion, loose coupling, and strict separation of concerns across audio synthesis, data modulation, and binary persistence.

1. **Data Abstraction:** The abstract base class `Oscillator` establishes a formal contract for periodic acoustic signal generators. Callers interact exclusively with

the pure virtual member function `nextSample()`, abstracting away underlying trigonometric evaluations, phase increments, and piecewise boundary calculations (Listing 24.1).

```
#pragma once

class Oscillator {
public:
    Oscillator(double frequency, double sampleRate);
    virtual ~Oscillator() = default;

    virtual double nextSample() = 0; // Pure virtual polymorphic interface

    void setFrequency(double frequency);
    void reset();

protected:
    void advancePhase();

    double frequency_;
    double sampleRate_;
    double phase_ = 0.0; // Normalized phase tracking [0.0, 1.0)
};
```

Listing 24.1: Abstract Oscillator Base Class (`include/Oscillator.h`)

2. **Polymorphism and Dynamic Dispatch:** The core synthesis pipeline in `render()` operates uniformly on an abstract base pointer `std::unique_ptr<Oscillator>`. Dynamic dispatch resolves invocations of `nextSample()` to the appropriate concrete waveform realization selected via a Factory Method pattern (Listing 24.2).

```
#include "SynthSettings.h"
#include "Envelope.h"
#include "Waveforms.h"
#include <memory>
#include <vector>

static std::unique_ptr<Oscillator> makeOscillator(const SynthSettings& s) {
    switch (s.waveform) {
        case Waveform::Square:
            return std::make_unique<SquareOscillator>(s.frequency, s.sampleRate);
        case Waveform::Sawtooth:
            return std::make_unique<SawtoothOscillator>(s.frequency, s.sampleRate);
        case Waveform::Sine:
        default:
            return std::make_unique<SineOscillator>(s.frequency, s.sampleRate);
    }
}

std::vector<double> render(const SynthSettings& s) {
    const std::size_t total = static_cast<std::size_t>(s.duration * s.sampleRate);
    std::unique_ptr<Oscillator> osc = makeOscillator(s);
```

```
    Envelope env(s.attack, s.decay, s.sustain, s.release, s.sampleRate);

    std::vector<double> samples;
    samples.reserve(total);

    for (std::size_t i = 0; i < total; ++i) {
        const double wave = osc->nextSample();
        const double gain = env.amplitudeAt(i, total);
        samples.push_back(wave * gain * s.amplitude);
    }
    return samples;
}
```

Listing 24.2: Polymorphic Factory and Rendering Pipeline (`src/Synth.cpp`)

3. **Encapsulation and Information Hiding:** Internal state attributes—such as accumulated phase and envelope parameters—are strictly encapsulated with `private` or `protected` access specifiers, mutating solely through validated member functions (Listing 24.3).

```
#include "Envelope.h"
#include <algorithm>

Envelope::Envelope(double attack, double decay, double sustain, double release,
                   double sampleRate)
    : attack_(attack), decay_(decay), sustain_(sustain), release_(release),
      sampleRate_(sampleRate) {}

double Envelope::amplitudeAt(std::size_t sampleIndex,
                             std::size_t totalSamples) const {
    const double t = static_cast<double>(sampleIndex) / sampleRate_;
    const double length = static_cast<double>(totalSamples) / sampleRate_;

    double gain;
    if (attack_ > 0.0 && t < attack_) {
        gain = t / attack_;
    } else if (decay_ > 0.0 && t < attack_ + decay_) {
        gain = 1.0 - (1.0 - sustain_) * ((t - attack_) / decay_);
    } else {
        gain = sustain_;
    }

    if (release_ > 0.0) {
        gain *= std::clamp((length - t) / release_, 0.0, 1.0);
    }
    return gain;
}
```

Listing 24.3: Encapsulated ADSR Dynamic Envelope Modulator (`src/Envelope.cpp`)

4. **Inheritance and Code Reusability:** The derived oscillator classes publicly inherit from `Oscillator`, directly reusing the underlying phase accumulator while

overriding concrete waveform transfer functions (Listing 24.4).

```cpp
#include "Waveforms.h"
#include <cmath>

namespace {
constexpr double kTwoPi = 6.28318530717958647692528676655900576;
}

double SineOscillator::nextSample() {
    const double sample = std::sin(kTwoPi * phase_);
    advancePhase();
    return sample;
}

double SquareOscillator::nextSample() {
    const double sample = (phase_ < 0.5) ? 1.0 : -1.0;
    advancePhase();
    return sample;
}

double SawtoothOscillator::nextSample() {
    const double sample = 2.0 * phase_ - 1.0;
    advancePhase();
    return sample;
}
```

Listing 24.4: Concrete Derived Waveform Implementations (`src/Waveforms.cpp`)

5. **Separation of Concerns (SoC):** The system isolates mathematical sound generation (`Oscillator`), amplitude modulation (`Envelope`), configuration parameters (`SynthSettings`), low-level binary serialization (`WavWriter`), and presentation frontends into decoupled compilation units.

24.3.1 Class Design and Associations

The relationships between the primary architectural components are structured as follows:

- **Generalization (Inheritance):** `SineOscillator`, `SquareOscillator`, and `SawtoothOscillator` publicly inherit from the abstract base class `Oscillator`, realizing the pure virtual interface `nextSample()`.
- **Composition:** The rendering pipeline (`render()`) instantiates and manages the lifecycle of an `Oscillator` (via `std::unique_ptr`) and an `Envelope` instance for the duration of the audio synthesis task.
- **Aggregation / Association:** The `SynthSettings` Plain Old Data (POD) structure aggregates configuration values alongside the strongly typed enumeration `Waveform`.
- **Dependency:** The static utility class `WavWriter` depends upon normalized `std::vector<double>` sample buffers produced by `render()` and interacts directly with `std::ofstream` binary sinks.

24.4 System Architecture and UML Modeling

The system adopts a decoupled three-tier architecture: the **User Interface Layer**, the **Core Synthesizer Engine** (`synth_core` static library), and the **Storage and Output Layer**. Figure 24.1 illustrates this high-level architectural decomposition.

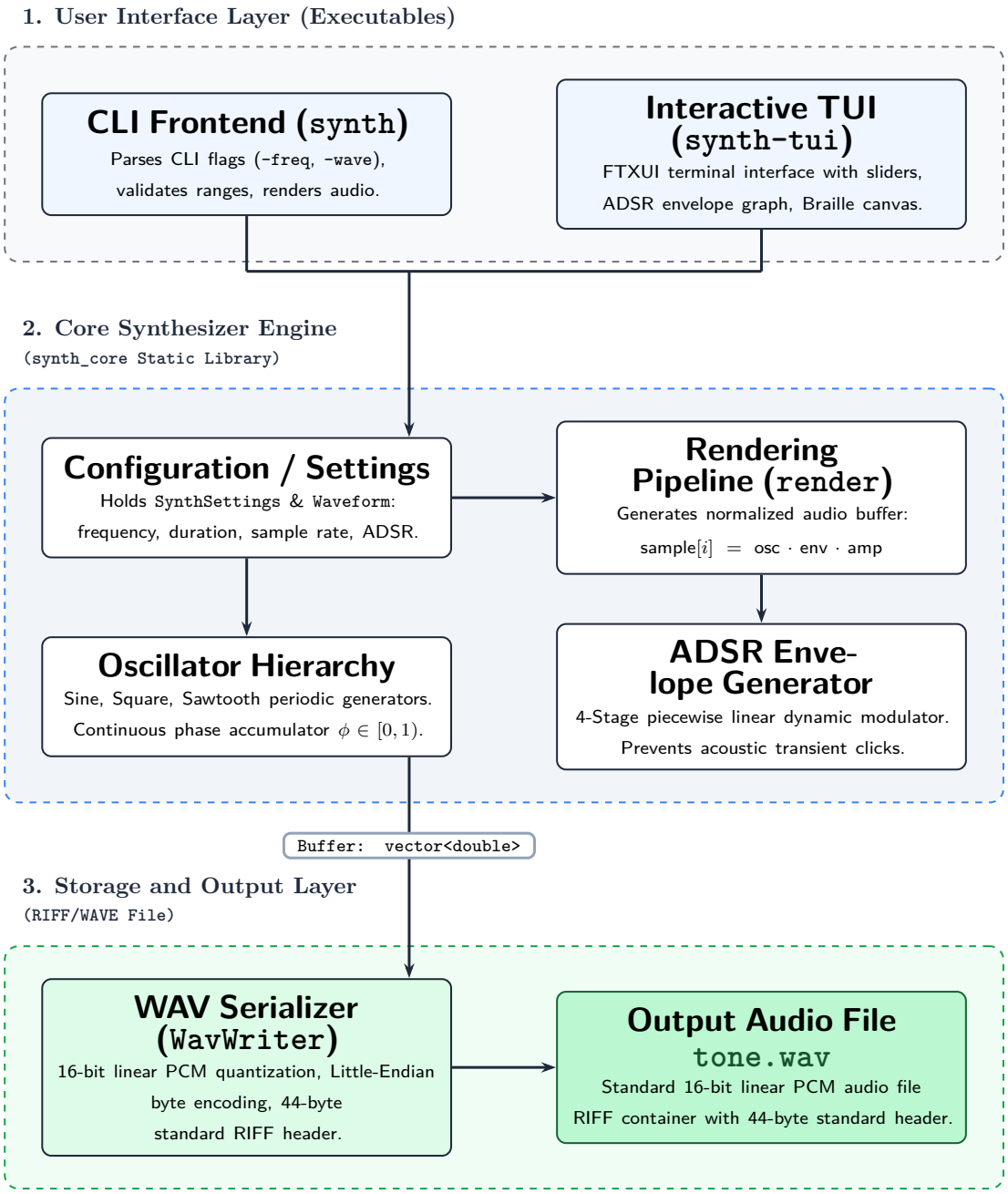


Figure 24.1: Three-tier decoupled architectural diagram of the Audio-WAV-Synthesizer.

24.4.1 UML Class Diagram

Figure 24.2 formally specifies the static structural model of the system, precisely matching the canonical object model. The abstract class `Oscillator` defines the virtual

interface `nextSample() = 0` and encapsulates normalized phase state ϕ. Concrete subclasses (`SineOscillator`, `SquareOscillator`, `SawtoothOscillator`) inherit phase tracking while providing distinct waveform evaluation logic.

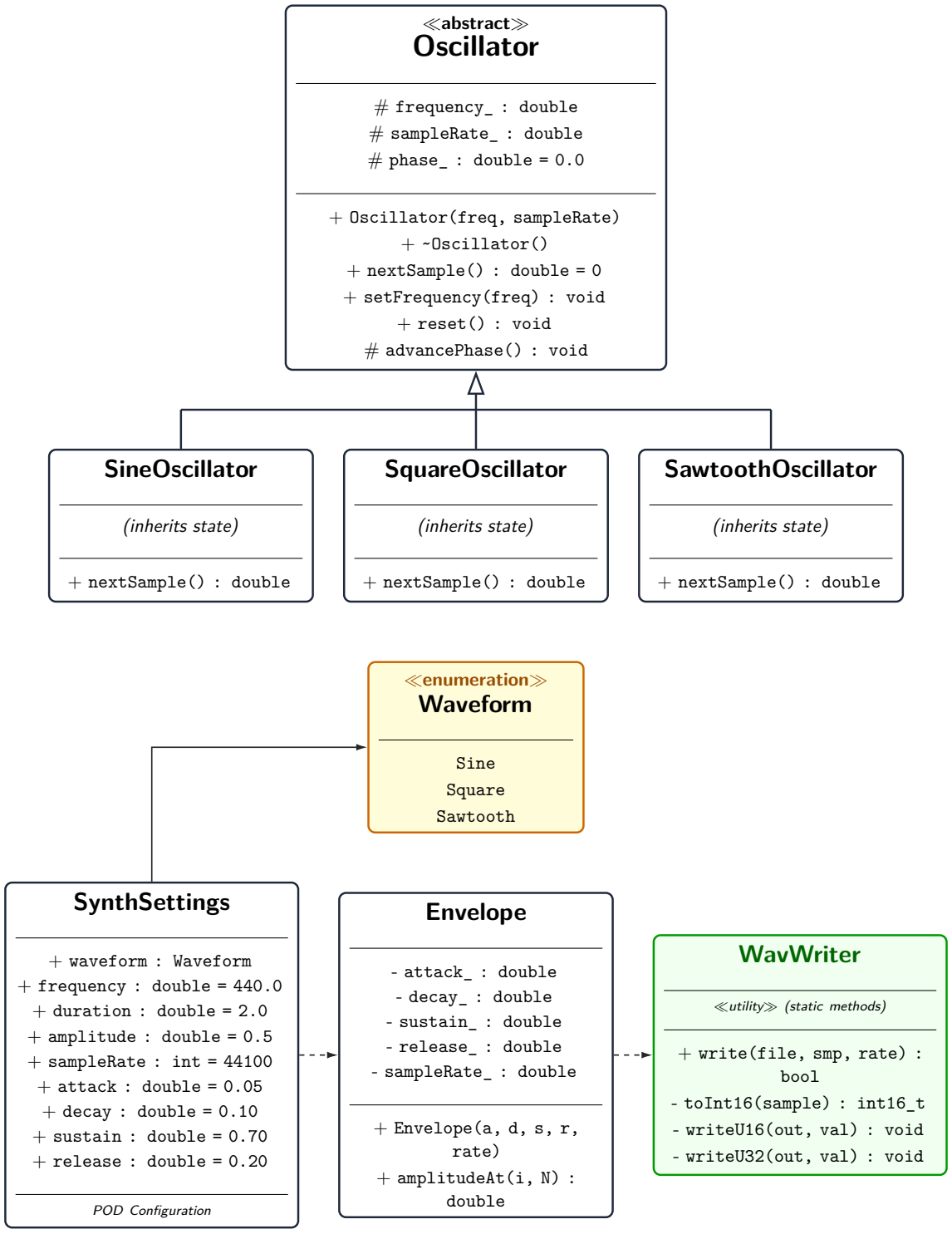


Figure 24.2: UML Class Diagram: Depicts the polymorphic base interface `Oscillator`, derived concrete waveforms, configuration encapsulation via `SynthSettings`, temporal envelope shaping, and static binary serialization.

24.5 Low-Level Binary RIFF/WAVE Serialization

Listing 24.5 documents the low-level binary serialization implementation. The static method quantizes floating-point samples to 16-bit signed integers ($[-32768, 32767]$) and formats the canonical 44-byte RIFF/WAVE header directly into an unbuffered binary `std::ofstream`.

```
#include "WavWriter.h"
#include <algorithm>
#include <cmath>
#include <fstream>

std::int16_t WavWriter::toInt16(double sample) {
    sample = std::clamp(sample, -1.0, 1.0);
    return static_cast<std::int16_t>(std::lround(sample * 32767.0));
}

void WavWriter::writeU16(std::ostream& out, std::uint16_t value) {
    const char bytes[2] = {
        static_cast<char>(value & 0xFF),
        static_cast<char>((value >> 8) & 0xFF)
    };
    out.write(bytes, 2);
}

void WavWriter::writeU32(std::ostream& out, std::uint32_t value) {
    const char bytes[4] = {
        static_cast<char>(value & 0xFF),
        static_cast<char>((value >> 8) & 0xFF),
        static_cast<char>((value >> 16) & 0xFF),
        static_cast<char>((value >> 24) & 0xFF)
    };
    out.write(bytes, 4);
}

bool WavWriter::write(const std::string& filename,
                      const std::vector<double>& samples,
                      std::uint32_t sampleRate,
                      std::uint16_t channels) {
    std::ofstream file(filename, std::ios::binary);
    if (!file) return false;

    const std::uint16_t bitsPerSample = 16;
    const std::uint16_t blockAlign = channels * bitsPerSample / 8;
    const std::uint32_t byteRate = sampleRate * blockAlign;
    const std::uint32_t dataSize = static_cast<std::uint32_t>(samples.
   size() * 2);
    const std::uint32_t chunkSize = 36 + dataSize;

    // 1. RIFF Container Header
    file.write("RIFF", 4);
    writeU32(file, chunkSize);
    file.write("WAVE", 4);

    // 2. Format Chunk ("fmt ")
    file.write("fmt ", 4);
    writeU32(file, 16);                 // Subchunk1Size (16 for PCM)
    writeU16(file, 1);                  // AudioFormat (1 = PCM
```

```
    uncompressed)
    writeU16(file, channels);          // NumChannels (1 = Mono)
    writeU32(file, sampleRate);        // SampleRate (e.g. 44100 Hz)
    writeU32(file, byteRate);          // ByteRate
    writeU16(file, blockAlign);        // BlockAlign
    writeU16(file, bitsPerSample);     // BitsPerSample (16 bits)

    // 3. Data Chunk ("data")
    file.write("data", 4);
    writeU32(file, dataSize);

    // 4. Quantized PCM Little-Endian Sample Streaming
    std::vector<char> buffer(samples.size() * 2);
    for (std::size_t i = 0; i < samples.size(); ++i) {
        const std::uint16_t v = static_cast<std::uint16_t>(toInt16(
    samples[i]));
        buffer[2 * i]     = static_cast<char>(v & 0xFF);
        buffer[2 * i + 1] = static_cast<char>((v >> 8) & 0xFF);
    }
    file.write(buffer.data(), static_cast<std::streamsize>(buffer.size
    ()));

    file.close();
    return !file.fail();
}
```

Listing 24.5: Raw Binary RIFF/WAVE File Serializer (`src/WavWriter.cpp`)

Table 24.2 summarizes the exact 44-byte structure assembled by `WavWriter`.

Offset	Field Name	Size	Endianness	Semantic Value / Role
0	`ChunkID`	4 Bytes	Big	`"RIFF"` ASCII signature
4	`ChunkSize`	4 Bytes	Little	$36 + \text{DataSize}$ (Total file size -8)
8	`Format`	4 Bytes	Big	`"WAVE"` container type
12	`Subchunk1ID`	4 Bytes	Big	`"fmt "` format chunk marker
16	`Subchunk1Size`	4 Bytes	Little	16 for uncompressed Linear PCM
20	`AudioFormat`	2 Bytes	Little	1 specifies Linear PCM
22	`NumChannels`	2 Bytes	Little	1 for Mono, 2 for Stereo
24	`SampleRate`	4 Bytes	Little	Sampling frequency f_s in Hz (e.g., 44100)
28	`ByteRate`	4 Bytes	Little	$\text{SampleRate} \times \text{Channels} \times \frac{\text{Bits}}{8}$
32	`BlockAlign`	2 Bytes	Little	$\text{Channels} \times \frac{\text{Bits}}{8}$ (2 for 16-bit mono)
34	`BitsPerSample`	2 Bytes	Little	16 bits quantization depth
36	`Subchunk2ID`	4 Bytes	Big	`"data"` payload identifier
40	`Subchunk2Size`	4 Bytes	Little	$N_{\text{samples}} \times \text{Channels} \times \frac{\text{Bits}}{8}$

Table 24.2: Canonical 44-Byte Microsoft RIFF/WAVE Header Specification

24.6 Conclusion

The Audio Synthesizer and WAV Generator project demonstrates the design of a decoupled, performant, and extensible software synthesizer implemented in modern C++17. By applying foundational Object-Oriented principles—including pure virtual polymorphic dispatch, dynamic inheritance hierarchies, strict state encapsulation, and modular

architectural separation—the application successfully bridges mathematical acoustics with systems-level binary file serialization.

Key milestones accomplished include:

- Developed an extensible abstract oscillator base hierarchy enabling straightforward addition of future waveform types.
- Eliminated audio click transients through a 4-stage piecewise linear ADSR envelope generator.
- Implemented direct low-level 16-bit little-endian RIFF/WAVE binary stream writing without external multimedia runtime dependencies.
- Provided dual frontends: a command-line interface for automated batch processing and an interactive terminal GUI powered by FTXUI.